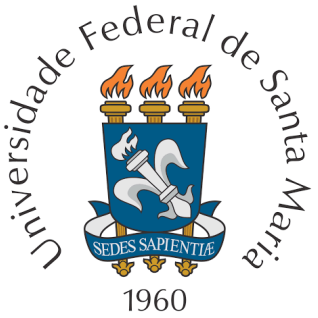


Universidade Federal de Santa Maria

Plano de Aula

Ana Julia Bock Medina
Eduarda Padilha Zorzo
Lorena Finamor Carvalho
Pedro Emilio Puntel Cassol

Santa Maria, 14 de out. de 2026

Plano de Aula

1. Identificação

Alunos: Ana Julia Bock Medina, Eduarda Padilha Zorzo, Lorena Finamor Carvalho e Pedro Emilio Puntel Cassol

Instituição: Universidade Federal de Santa Maria, Santa Maria – RS.

Disciplina: Engenharia de Software II

Curso: Sistemas para Internet

Tema: Docker e ambientes de execução: desenvolvimento, homologação e produção

Ano: 2026 **Semestre:** 2

2. Ementa

Fundamentos da conteinerização e do Docker. Problemas de inconsistência entre ambientes de execução. Imagens, camadas, tags, containers, Docker Engine, Docker Hub e Dockerfile. Construção e execução de imagens e containers, comandos básicos e Docker Swarm. Utilização do Docker nos ambientes de desenvolvimento, homologação e produção. Configurações, redes, portas, bancos de dados, volumes e Docker Compose. Integração e entrega contínuas (CI/CD), versionamento e rollback. Demonstração prática, vantagens, limitações e boas práticas de segurança no uso de containers.

3. Público-alvo e pré-requisitos

Esta aula é voltada principalmente para os alunos do curso de Sistemas para Internet, com foco na utilização do Docker no desenvolvimento de aplicações e na execução dessas aplicações em diferentes ambientes.

Como pré-requisitos, espera-se que os alunos possuam conhecimentos básicos de informática, desenvolvimento de software, arquivos e sistemas operacionais. Noções básicas de programação e utilização de terminal podem auxiliar no acompanhamento dos exemplos apresentados. Entretanto, não é necessário possuir conhecimentos prévios sobre Docker, pois a aula apresenta seus conceitos fundamentais, comandos básicos e aplicações práticas de forma gradual.

O conteúdo aborda desde os problemas de inconsistência entre ambientes de execução até a utilização do Docker nos ambientes de desenvolvimento, homologação e produção, incluindo a construção e execução de containers, o gerenciamento de dados e as boas práticas de segurança.Objetivos

Esta aula objetiva apresentar os fundamentos do Docker e sua aplicação no desenvolvimento e na execução de aplicações em diferentes ambientes. Ao fim da aula, espera-se que o aluno tenha condições de:

- Compreender os problemas de inconsistência entre os ambientes de desenvolvimento, homologação e produção;
- Entender os conceitos fundamentais do Docker, diferenciando imagens, containers, Docker Engine, Docker Hub e Dockerfile;
- Compreender o processo de construção de imagens e execução de containers, reconhecendo os principais comandos do Docker;
- Identificar como o Docker pode ser utilizado nos ambientes de desenvolvimento, homologação e produção, incluindo configurações, redes, portas e persistência de dados;
- Conhecer ferramentas e recursos relacionados ao gerenciamento de aplicações com Docker, como Docker Compose, Docker Swarm e práticas de CI/CD;
- Reconhecer as vantagens, limitações e boas práticas de segurança relacionadas à utilização de containers;
- Acompanhar e compreender a demonstração prática de criação de imagens e execução de containers Docker.

4. Duração

A aula terá duração mínima de 1 hora e 30 minutos, contemplando a apresentação dos conceitos fundamentais do Docker, sua utilização nos ambientes de desenvolvimento, homologação e produção, a demonstração prática e a realização de uma atividade desplugada.

5. Conteúdos abordados

Os conteúdos abordados são baseados na ementa da aula:

1. Ambientes de execução

1. Desenvolvimento, homologação e produção;
2. Problemas de inconsistência entre ambientes;
3. Padronização, consistência, isolamento e reprodutibilidade.

2. Fundamentos do Docker

1. Contexto histórico e surgimento do Docker;
2. Conceito de Docker e conteinerização;
3. Diferenças entre máquinas virtuais e containers.

3. Imagens e containers

1. Conceito de imagem Docker;
2. Docker Hub;
3. Camadas, tags e versões de imagens;
4. Conceito de container e Docker Engine;
5. Docker Swarm.

4. Construção e execução de containers

1. Dockerfile e construção de imagens;
2. Relação entre Dockerfile, imagem e container;
3. Principais comandos do Docker;
4. Demonstração prática de criação de imagens e execução de containers.

5. Docker nos ambientes de execução

1. Utilização do Docker em desenvolvimento, homologação e produção;
2. Construção, publicação e utilização de imagens versionadas;
3. Configurações e variáveis de ambiente;
4. Arquivo .dockerignore;
5. Redes, portas e comunicação entre containers;
6. Bancos de dados e persistência de dados por meio de volumes;
7. Utilização do Docker Compose;
8. Integração e entrega contínuas (CI/CD) e rollback.

6. Vantagens, limitações e segurança

1. Vantagens e limitações do uso do Docker;
2. Boas práticas de segurança;
3. Proteção de senhas e informações sensíveis;
4. Atualização de imagens e dependências;
5. Controle de privilégios, exposição de portas, autenticação, HTTPS e backups.

7. Atividade prática e fixação do conteúdo

1. Aula 1 de introdução a Docker, com foco em dockerfile;
2. Aula 2 de aprofundamento, com noções de volumes, Docker Compose, banco de dados e persistência;
3. Abordagem extra em persistência de dados, sincronização e centralização de forma teórica.

6. Metodologia

A aula será desenvolvida por meio de momentos expositivos e práticos, combinando a apresentação dos conceitos teóricos com exemplos de utilização do Docker. Ao final, será realizada uma atividade desplugada no formato de um jogo de perguntas e respostas inspirado no programa *Show do Milhão*, com o objetivo de revisar os conteúdos abordados, estimular a participação dos alunos e verificar a compreensão dos conceitos apresentados durante a aula.

A organização da aula é dada como segue:

- **Parte teórica**
    - Apresentação com projetor:

        - Apresentação do tema e dos objetivos da aula;
        - Introdução ao problema da inconsistência entre ambientes de execução;
        - Apresentação dos conceitos fundamentais do Docker;
        - Explicação das diferenças entre máquinas virtuais e containers;
        - Apresentação de imagens, containers, Docker Engine, Docker Hub, Dockerfile e Docker Swarm;
        - Explicação dos comandos básicos e do processo de criação de imagens e execução de containers;
        - Abordagem da utilização do Docker nos ambientes de desenvolvimento, homologação e produção;
        - Apresentação de configurações, redes, portas, bancos de dados, volumes, Docker Compose e CI/CD;
        - Aprofundamento teórico de persistência de dados com Docker, sincronização e centralização;
        - Discussão sobre as vantagens, limitações e boas práticas de segurança.
- **Parte prática**
    - Apresentação pelo computador utilizando o Intellij IDEA e navegador:
        - Aplicação prática dos conteúdos abordados na prática;
        - Primeira aula dedicada à introdução à Docker, com explicações principalmente sobre Dockerfile, e como funciona a criação de imagens, containers e como acessar a aplicação;
        - Segunda aula de aprofundamento em dados, com volumes e docker compose, explicando sobretudo persistência de dados de forma superficial;
        - Ao longo das aulas serão abordados analogias como: bolo, receita, cozinha, etc, como forma de facilitar o entendimento do assunto.
- **Atividade desplugada**
    - Realização da dinâmica Show do Milhão — Docker:
        - Organização dos alunos em equipes para a realização da dinâmica;
        - Apresentação das regras do jogo e da tabela de pontuação;
        - Leitura de perguntas relacionadas aos conteúdos abordados durante a aula, divididas em quatro níveis de dificuldade: Fácil, Médio, Difícil e Milhão;
        - Concessão do direito de resposta à primeira equipe que levantar a mão após a leitura de cada pergunta;
        - Atribuição de pontos para respostas corretas e desconto de pontos para respostas incorretas, conforme os valores estabelecidos para cada nível;

- Acumulação da pontuação ao longo das rodadas, com o objetivo de identificar a equipe vencedora ao final das 16 perguntas;
- Realização de quatro perguntas por nível de dificuldade, totalizando 16 perguntas, sendo as quatro últimas pertencentes ao Nível Milhão, no qual cada acerto ou erro resulta, respectivamente, em um ganho ou perda de 1.000.000 de pontos;
- Divulgação da equipe vencedora, considerando a maior pontuação acumulada ao final da atividade.
- As perguntas e suas respectivas alternativas estão disponíveis no **Anexo I — Perguntas e Alternativas**.
- O gabarito comentado de cada questão está disponível no **Anexo II - Gabarito Comentado**.

7. Recursos Didáticos

O conteúdo será apresentado aos alunos por meio de uma apresentação de slides, com o auxílio de projetor ou equipamento multimídia. Durante a parte teórica, não será necessário que os alunos utilizem computadores, pois os conceitos serão explicados por meio de recursos visuais e exemplos apresentados nos slides. Para a demonstração prática, será necessário um computador com o Docker instalado e configurado, preferencialmente com acesso à Internet, para a apresentação dos comandos e da execução de containers.

Será utilizado o seguinte material de apoio:

- Apresentação de slides com os conceitos teóricos, exemplos e comandos básicos do Docker;
- Computador com Docker instalado para a demonstração prática;
- Projetor ou equipamento multimídia para a exposição do conteúdo;

8. Referências

DATACAMP. **Docker para iniciantes: um guia prático para contêineres**. [S. l.], 24 fev. 2025. Disponível em: https://www.datacamp.com/pt/tutorial/docker-tutorial. Acesso em: 07 out. 2026.

INSIGHT DATA SCIENCE LAB. **Introdução ao Docker**. [S. l.], [2020?]. Disponível em: https://insightlab.ufc.br/wp-content/uploads/2020/05/Introdução-a-Docker-compactado.pdf. Acesso em: 07 out. 2026.

AMAZON WEB SERVICES (AWS). **Qual é a diferença entre imagens e contêineres do Docker?** [S. l.], [s. d.]. Disponível em:

https://aws.amazon.com/pt/compare/the-difference-between-docker-images-and-containers/. Acesso em: 09 out. 2026.

COD3R. **Apostila Docker**. [S. l.], [s. d.]. Disponível em: http://files.cod3r.com.br/apostila-docker.pdf. Acesso em: 12 out. 2026.

DOCKER. **Docker Documentation**. Disponível em: https://docs.docker.com/. Acesso em: 8 out. 2026.

DOCKER. **Docker Compose**. Disponível em: https://docs.docker.com/compose/. Acesso em: 8 out. 2026.

DOCKER. **What is a container?** Disponível em: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/. Acesso em: 10 out. 2026.

DOCKER INC. Docker Docs. 2026. Disponível em: https://docs.docker.com/. Acesso em: 11 out. 2026.

HUMBLE, J.; FARLEY, D. Continuous Delivery: reliable software releases through build, test, and deployment automation. Upper Saddle River: Addison-Wesley, 2010.

WIGGINS, A. The Twelve-Factor App. 2011. Disponível em: https://12factor.net/pt_br/. Acesso em: 12 out. 2026

Anexo I — Perguntas e Alternativas

Nível fácil

1. **Qual é a finalidade principal do ambiente de homologação?**

   A) Desenvolver o código pela primeira vez
   B) Validar a aplicação antes de disponibilizá-la aos usuários
   C) Substituir o desenvolvimento
   D) Sincronizar os dados

2. **Durante o desenvolvimento de uma aplicação, um aluno precisa construir uma imagem Docker e iniciar um container. Qual alternativa descreve corretamente cada componente?**
   a) O Docker Hub constrói imagens localmente, o Dockerfile inicia containers e o Engine armazena imagens online.
   b) O Dockerfile define as instruções da imagem, o Engine gerencia as operações do Docker e o Hub permite compartilhar e baixar imagens.
   c) O Docker Engine contém o código-fonte, o Hub executa containers e o Dockerfile gerencia os processos em execução.
   d) O Dockerfile gerencia containers, o Engine funciona como catálogo de imagens e o Hub define as instruções de construção.

3. **Que arquivo impede que o .env com as senhas seja copiado para dentro da imagem durante o build?**
   a) .gitignore
   b) compose.yaml
   c) .dockerignore
   d) Dockerfile

4. **Qual sequência representa corretamente a criação e execução de uma aplicação com Docker?**

   a) Aplicação → container → Dockerfile → imagem → `docker build`
   b) Dockerfile → `docker run` → imagem → `docker build` → container
   c) Aplicação → Dockerfile → `docker build` → imagem → `docker run` → container
   d) Aplicação → imagem → Dockerfile → container → `docker run`

Nível médio

1. **Um sistema utiliza MySQL no desenvolvimento e SQLite no ambiente de produção. Qual afirmação é mais correta?**

   A) Toda consulta MySQL falhará no SQLite
   B) Os dois bancos são exatamente iguais
   C) A aplicação sempre funcionará, pois ambos usam SQL
   D) Algumas consultas ou funcionalidades podem ser incompatíveis entre os bancos

2. **Uma equipe utiliza as imagens `python:3.12-slim` e `nginx:alpine` em seus projetos. Considerando a estrutura e a identificação das imagens Docker, qual afirmação está correta?**

   a) As palavras `slim` e `alpine` identificam containers em execução, enquanto `3.12` e `nginx` indicam as camadas utilizadas durante a execução.
   b) Cada tag representa uma camada independente da imagem, e imagens com tags diferentes não podem reaproveitar camadas em comum.
   c) As tags identificam versões ou variantes das imagens, enquanto as camadas compõem a imagem e podem ser reaproveitadas entre imagens compatíveis.
   d) As tags identificam exclusivamente a linguagem de programação utilizada, enquanto as camadas armazenam somente os arquivos temporários gerados pelo container.

3. **Se a imagem é a mesma em todos os ambientes, como a aplicação sabe qual banco usar e qual nível de log aplicar?**
   a) Existe uma versão do código para cada ambiente
   b) Pelas variáveis de ambiente definidas em cada ambiente
   c) O Docker detecta sozinho em que servidor está rodando
   d) A configuração fica gravada dentro da imagem no build

4. **No comando abaixo, qual é a função de `-p 8080:80`?**

   docker run -d -p 8080:80 app_aula:1.0

   a) Conectar a porta 8080 do container à porta 80 do PostgreSQL.
   b) Encaminhar a porta 8080 do computador para a porta 80 do container.
   c) Executar 80 containers usando a porta 8080.
   d) Alterar a porta interna do Nginx de 80 para 8080.

Nível difícil

1. **Uma equipe padronizou as dependências com Docker, mas a aplicação ainda falha porque a senha do banco está incorreta. O que isso demonstra?**

   A) Docker não tem utilidade
   B) Docker ajuda a padronizar o ambiente, mas não corrige automaticamente todas as configurações externas
   C) Senhas não influenciam conexões
   D) O código precisa sempre todo reescrito

2. **Uma empresa deseja executar várias aplicações isoladas em um mesmo servidor, reduzindo o consumo de recursos em comparação com máquinas virtuais tradicionais. Qual alternativa explica melhor como os containers Docker podem ajudar nesse cenário?**

   a) Cada container executa um sistema operacional convidado completo, mas dispensa o uso de recursos do sistema operacional da máquina hospedeira.
   b) Os containers compartilham o kernel do sistema operacional hospedeiro e mantêm isolamento entre aplicações e seus ambientes, geralmente evitando a sobrecarga de um sistema operacional completo por container.
   c) O Docker Engine cria um kernel independente para cada container, garantindo que nenhuma biblioteca ou dependência seja compartilhada entre aplicações.
   d) Os containers eliminam a necessidade de isolamento entre processos, utilizando diretamente todas as bibliotecas instaladas no sistema operacional hospedeiro.

3. **No desenvolvimento, o código chega ao container por bind mount ou Compose Watch. E em homologação e produção?**
   a) Também por bind mount, apontando para a pasta do servidor
   b) O código é baixado do GitHub toda vez que o container sobe
   c) O Compose Watch sincroniza o código do desenvolvedor com o servidor
   d) O código vai dentro da própria imagem

4. **Uma embarcação foi cadastrada no PostgreSQL usando um volume nomeado. Depois, foram executados:**

   docker compose down
   docker compose up -d

O que deve acontecer com o cadastro?

a) Será apagado, pois down sempre remove containers e volumes.
b) Continuará salvo, pois o volume nomeado é preservado e reutilizado.
c) Será copiado para a imagem da aplicação durante o novo up.
d) Continuará salvo apenas se o container antigo possuir o mesmo ID.

Nível milhão

1. **Qual afirmação conecta corretamente a história do Docker aos problemas discutidos na apresentação?**

   A) O Docker surgiu em 2013 e contribuiu para tornar ambientes mais consistentes, mas não inventou os containers nem eliminou todos os problemas.
   B) O Docker foi criado em 2014 e criou a tecnologia de containers para garantir consistência.
   C) O Docker criou os ambientes de desenvolvimento, homologação e produção.
   D) O Docker garante que uma aplicação sempre funcione em qualquer sistema operacional sem necessidade de configurações.

2. **Uma empresa utiliza Docker Swarm para gerenciar containers em várias máquinas, controlar réplicas e realizar atualizações. Qual alternativa descreve corretamente o funcionamento do Swarm?**
   a) O manager constrói e armazena imagens, enquanto os workers definem e distribuem as réplicas.
   b) O manager coordena o cluster e os serviços, enquanto os workers executam tarefas. O Swarm permite gerenciar réplicas e atualizações.
   c) Cada worker gerencia suas próprias réplicas, enquanto o manager apenas monitora os nós, sem gerenciar serviços.
   d) O manager coordena os serviços, mas cada container precisa de um sistema operacional completo, como nas máquinas virtuais.

3. **A versão app:1.4.0 foi para produção e apareceu um erro grave. Qual é a atitude correta?**
   a) Entrar no container de produção e corrigir o código direto nele
   b) Trocar a tag para latest para pegar a versão mais nova
   c) Rodar docker build no servidor de produção com a correção
   d) Fazer um rollback, subindo a imagem anterior para a produção continuar funcionando normalmente
4. **Duas pessoas executam o mesmo `compose.yaml` em computadores diferentes. Elas querem compartilhar os dados e visualizar alterações**

sem atualizar manualmente a página. Qual solução está tecnicamente correta?

a) Criar volumes com o mesmo nome nos dois computadores, pois nomes iguais sincronizam os dados.
b) Compartilhar somente a imagem da aplicação, pois imagens também armazenam os novos cadastros.
d) Usar uma aplicação e um PostgreSQL centrais, fazendo todos acessarem o mesmo servidor, e implementar consultas periódicas, Server-Sent Events ou WebSocket para atualização automática das telas.
d) Publicar diretamente o PostgreSQL para todos os usuários, eliminando a aplicação e o controle de acesso.

Anexo II - Gabarito Comentado

Nível fácil

1. Resposta: B - Homologação permite validar a aplicação antes de sua liberação para uso.

2. Resposta: B - Cada componente tem uma função diferente: o Dockerfile define como construir a imagem, o Engine gerencia as operações e o Hub permite encontrar e compartilhar imagens.

3. Resposta: C - O .dockerignore funciona como o .gitignore, só que para o build: o que estiver listado nele não é enviado ao Docker. Sem ele, um COPY levaria o .env com as senhas para dentro da imagem.

4. Resposta: C - O Dockerfile descreve a construção. `docker build` transforma essas instruções em uma imagem. `docker run` utiliza a imagem para criar e executar um container.

Nível médio

1. **Resposta: D -** Embora ambos utilizem SQL, o MySQL e o SQLite possuem dialetos, funções e tipos de dados diferentes.

2. Resposta: C - As tags ajudam a identificar qual imagem utilizar, enquanto as camadas formam a imagem e podem ser reutilizadas.

3. Resposta: B - A imagem é a mesma, mas a configuração vem de fora dela. Cada ambiente define suas próprias variáveis, como LOG_LEVEL e DATABASE_URL

4. Resposta: B - A ordem é:

HOST:CONTAINER

8080:80

O navegador acessa a porta 8080 do computador, e o Docker encaminha a requisição para a porta 80 do container.

Nível difícil

1. **Resposta B** - A padronização do ambiente não garante que credenciais ou serviços externos estejam corretos

2. Resposta B - Diferentemente das máquinas virtuais tradicionais, os containers compartilham o kernel do sistema hospedeiro, o que geralmente os torna mais leves.

3. Resposta: D - Bind mount e Compose Watch servem para ver na hora as alterações feitas no editor, sem reconstruir a imagem a cada mudança. Fora do desenvolvimento, o código vai empacotado na imagem, porque é ela que foi testada na homologação e segue igual até a produção.

4. Resposta: B - `docker compose down` remove os containers e a rede, mas preserva volumes nomeados por padrão. Quando os serviços são recriados, o PostgreSQL volta a utilizar o mesmo volume. O volume seria removido se fosse utilizado:

docker compose down -v

Nível milhão

1. **Resposta: A -** Essa alternativa condiz com o contexto histórico e os limites reais do Docker.

2. **Resposta: B** - O Docker Swarm organiza e gerencia containers distribuídos em várias máquinas, permitindo controlar serviços e suas réplicas.

3. Resposta: D - Como cada versão tem uma tag fixa no registro, basta subir de novo a imagem anterior (por exemplo, a 1.3.2) para a produção voltar a funcionar. Depois, a correção é feita no código e entra como uma nova imagem versionada.

4. Resposta: C - Volumes comuns não são sincronizados entre computadores. Para compartilhar dados, todos devem acessar uma aplicação central conectada ao mesmo PostgreSQL.
Mesmo com os dados centralizados, as telas não mudam automaticamente. Atualização em tempo real exige outro mecanismo, como consultas periódicas, Server-Sent Events ou WebSocket.